

Работа с Arduino и MPU6050

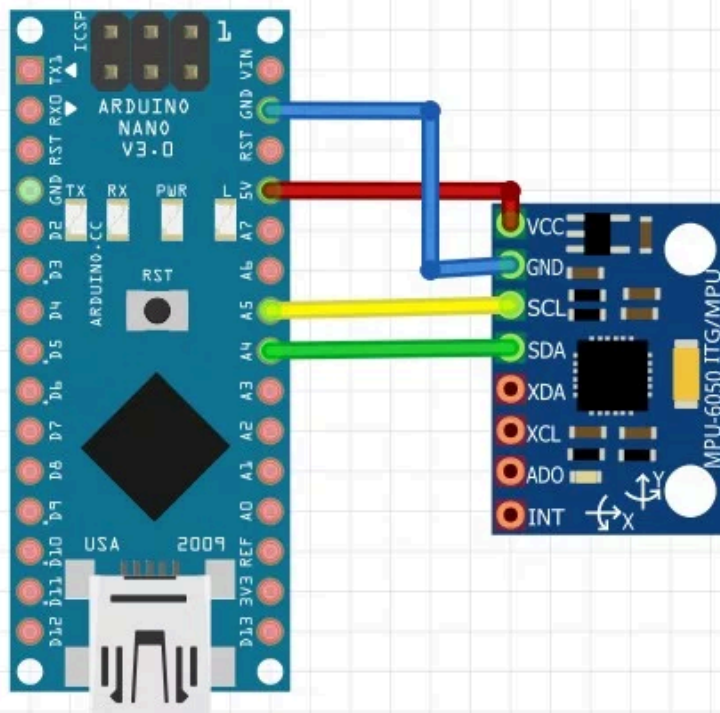
[внешний источник](#)

MPU6050 представляет собой 3-х осевой гироскоп и 3-х же осевой акселерометр в одном корпусе. Сразу поясню, что это и для чего:

- Гироскоп измеряет **угловую скорость вращения вокруг оси**, условно в градусах/секунду. Если датчик лежит на столе – по всем трём осям будет значение около нуля. Для нахождения текущего угла по скорости нужно интегрировать эту скорость. Гироскоп может чуть привирать, поэтому ориентироваться на него для расчёта текущего угла не получится даже при идеальной калибровке.
- Акселерометр измеряет **ускорение вдоль оси**, условно в метрах/секунду/секунду. Если датчик лежит на столе или движется с постоянной скоростью – на оси будет спроецирован вектор силы тяжести. Если датчик движется с ускорением – вдобавок к ускорению свободного падения получим составляющие вектора ускорения. Если датчик находится в свободном падении (в том числе на орбите планеты) – величины ускорений по всем осям будут равны 0. Зная проекции вектора силы тяжести можно с высокой точностью определить угол наклона датчика относительно него (привет, школьная математика). Если датчик движется – однозначно определить направление вектора силы тяжести не получится, соответственно угол тоже.

Подключение

Датчик подключается на шину i2c (**SDA -> A4, SCL -> A5, GND -> GND**). На плате стоит стабилизатор, позволяющий питаться от пина 5V (**VCC -> 5V**).



На модуле выведен пин **AD0**. Если он никуда не подключен (или подключен к **GND**) – адрес датчика на шине i2c будет 0x68, если подключен к питанию (**VCC**) – адрес будет 0x69. Таким образом без дополнительных микросхем можно подключить два датчика на шину с разными адресами.

Получение сырых данных

С MPU6050 можно общаться напрямую при помощи встроенной библиотеки *Wire.h* и даташита, а можно взять очень мощную библиотеку от i2cdev: [официальный сайт](#) (на момент написания не работает) или [GitHub](#). Для работы понадобится сама библиотека MPU6050 и библиотека I2Cdev (удобного способа скачать там нет, поэтому выложил архив с обеими на [Яндекс.Диск](#)). Мы будем работать с библиотекой, но я оставлю пример опроса всех ускорений и угловых скоростей напрямую, т.к. библиотека тяжёлая, а пример очень лёгкий.

Опрос в ручную

```
#include "Wire.h"
const int MPU_addr = 0x68; // адрес датчика
// массив данных
// [accX, accY, accZ, temp, gyrX, gyrY, gyrZ]
// асс - ускорение, gyr - угловая скорость, temp - температура (raw)
int16_t data[7];
void setup() {
    // инициализация
    Wire.begin();
    Wire.beginTransmission(MPU_addr);
    Wire.write(0x6B); // PWR_MGMT_1 register
    Wire.write(0);    // set to zero (wakes up the MPU-6050)
    Wire.endTransmission(true);

    Serial.begin(9600);
}
void loop() {
    getData(); // получаем
    // выводим
    for (byte i = 0; i < 7; i++) {
        Serial.print(data[i]);
        Serial.print('\t');
    }
    Serial.println();
    delay(200);
}
void getData() {
    Wire.beginTransmission(MPU_addr);
    Wire.write(0x3B); // starting with register 0x3B (ACCEL_XOUT_H)
    Wire.endTransmission(false);
    Wire.requestFrom(MPU_addr, 14, true); // request a total of 14 registers
    for (byte i = 0; i < 7; i++) {
        data[i] = Wire.read() << 8 | Wire.read();
    }
}
```

Опрос через библиотеку i2cdev

```
#include "MPU6050.h"
MPU6050 mpu;
int16_t ax, ay, az;
int16_t gx, gy, gz;
void setup() {
    Wire.begin();
    Serial.begin(9600);
    mpu.initialize();
    // состояние соединения
    Serial.println(mpu.testConnection() ? "MPU6050 OK" : "MPU6050 FAIL");
    delay(1000);
}
void loop() {
    mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
    Serial.print(ax); Serial.print('\t');
    Serial.print(ay); Serial.print('\t');
    Serial.print(az); Serial.print('\t');
    Serial.print(gx); Serial.print('\t');
    Serial.print(gy); Serial.print('\t');
    Serial.println(gz);
    delay(5);
}
```

Библиотека MPU6050 от i2cdev

Эта библиотека является одной из самых сложных в Ардуино-среде, да и сам датчик чего стоит — 50 страниц даташита + 50 страниц с описанием регистров и настроек... А в библиотеке — несколько сотен (!) функций для работы с модулем и его многочисленными настройками.

Рассмотрим самые «нужные», которые точно пригодятся в работе:

```
// ===== ГИРОСКОП =====
// получить угловые скорости по осям
int16_t getRotationX();
int16_t getRotationY();
int16_t getRotationZ();
void getRotation(int16_t* x, int16_t* y, int16_t* z);

// получить диапазон гироскопа
// 0 = +/- 250 degrees/sec
// 1 = +/- 500 degrees/sec
// 2 = +/- 1000 degrees/sec
// 3 = +/- 2000 degrees/sec
uint8_t getFullScaleGyroRange();

// установить диапазон гироскопа в формате
// MPU6050_GYRO_FS_250
```

```
// MPU6050_GYRO_FS_500
// MPU6050_GYRO_FS_1000
// MPU6050_GYRO_FS_2000
void setFullScaleGyroRange(uint8_t range);

// ===== АКСЕЛЕРОМЕТР =====
// получить ускорения по осям
int16_t getAccelerationX();
int16_t getAccelerationY();
int16_t getAccelerationZ();
void getAcceleration(int16_t* x, int16_t* y, int16_t* z);

// получить диапазон акселя
// 0 = +/- 2g
// 1 = +/- 4g
// 2 = +/- 8g
// 3 = +/- 16g
uint8_t getFullScaleAccelRange();

// установить диапазон акселя в формате
// MPU6050_ACCEL_FS_2
// MPU6050_ACCEL_FS_4
// MPU6050_ACCEL_FS_8
// MPU6050_ACCEL_FS_16
void setFullScaleAccelRange(uint8_t range);

// ===== ВСЯКОЕ =====
// получить ускорение, угловую скорость
void getMotion6(int16_t* ax, int16_t* ay, int16_t* az, int16_t* gx, int16_t* gy,
int16_t* gz);

// получить температуру
int16_t getTemperature();

// перезагрузить
void reset();

// управление режимом сна
bool getSleepEnabled();
void setSleepEnabled(bool enabled);

// ===== ПРЕРЫВАНИЕ СВОБОДНОГО ПАДЕНИЯ =====
// порог датчика свободного падения 0..255. Одна единица - 2mg
uint8_t getFreefallDetectionThreshold(); // получить порог
void setFreefallDetectionThreshold(uint8_t threshold); // установить порог

// время определения свободного падения 0..255. Одна единица - 1мс
uint8_t getFreefallDetectionDuration(); // получить время
void setFreefallDetectionDuration(uint8_t duration); // установить время
```

```

// прерывание
bool getIntFreefallEnabled();
void setIntFreefallEnabled(bool enabled);

// ===== ПРЕРЫВАНИЕ ДВИЖЕНИЯ =====
// порог датчика движения 0..255. Одна единица - 2mg
uint8_t getMotionDetectionThreshold();           // получить порог
void setMotionDetectionThreshold(uint8_t threshold); // установить порог

// время определения движения 0..255. Одна единица - 1мс
uint8_t getMotionDetectionDuration();           // получить время
void setMotionDetectionDuration(uint8_t duration); // установить время

// прерывание
bool getIntMotionEnabled();
void setIntMotionEnabled(bool enabled);

// ===== ПРЕРЫВАНИЕ ОТСУТСТВИЯ ДВИЖЕНИЯ =====
// порог определения отсутствия движения 0..255. Одна единица - 2mg
uint8_t getZeroMotionDetectionThreshold();           // получить порог
void setZeroMotionDetectionThreshold(uint8_t threshold); // установить порог

// время определения отсутствия движения 0..255. Одна единица - 1мс
uint8_t getZeroMotionDetectionDuration();           // получить время
void setZeroMotionDetectionDuration(uint8_t duration); // установить время

// прерывание
bool getIntZeroMotionEnabled();
void setIntZeroMotionEnabled(bool enabled);

// ===== ПРЕРЫВАНИЯ =====
// включить/выключить прерывания
bool getInterruptMode();
void setInterruptMode(bool mode);

// байт со всеми прерываниями. Настройка прерываний
uint8_t getIntEnabled();
void setIntEnabled(uint8_t enabled);

// статусы прерываний
uint8_t getIntStatus();
bool getIntFreefallStatus();
bool getIntMotionStatus();
bool getIntZeroMotionStatus();

// ===== ОФФСЕТЫ =====
// установить. XYZ аксель, XYZ гиро
void setXAccelOffset(int16_t offset);
void setYAccelOffset(int16_t offset);

```

```

void setZAccelOffset(int16_t offset);
void setXGyroOffset(int16_t offset);
void setYGyroOffset(int16_t offset);
void setZGyroOffset(int16_t offset);

// получить. XYZ аксель, XYZ гиро
int16_t getXAccelOffset();
int16_t getYAccelOffset();
int16_t getZAccelOffset();
int16_t getXGyroOffset();
int16_t getYGyroOffset();
int16_t getZGyroOffset();

// вывести оффсеты в порт
void PrintActiveOffsets();

void CalibrateGyro(uint8_t Loops = 15); // калибровать гиро(кол-во итераций)
void CalibrateAccel(uint8_t Loops = 15); // калибровать аксель(кол-во итераций)

```

Расшифровка сырых данных

По каждой оси и параметру датчик выдаёт 16-битное знаковое значение (

-32768.. 32767

). При стандартных настройках (как в примерах выше) это значение отражает:

- Ускорение в диапазоне -2.. 2 g (9.82 м/с/с).
- Угловую скорость в диапазоне -250.. 250 градусов/секунду.

Таким образом, для перевода сырых данных в величины СИ (**если это нужно**) можно сделать по каждой оси:

- Ускорение в м/с/с при чувствительности **2**: $\text{float accX_f} = \text{accX} / 32768 * 2$
- Угловая скорость в град/с при чувствительности **250**: $\text{float gyrX_f} = \text{gyrX} / 32768 * 250$

Настройка чувствительности

Датчик позволяет настроить чувствительность по каждому параметру, в библиотеке i2cdev это делается так:

- `setFullScaleAccelRange(MPU6050_ACCEL_FS_2);`
— диапазон -2.. 2 g
- `setFullScaleAccelRange(MPU6050_ACCEL_FS_4);`
— диапазон -4.. 4 g

- `setFullScaleAccelRange(MPU6050_ACCEL_FS_8);`
— диапазон -8.. 8 g
- `setFullScaleAccelRange(MPU6050_ACCEL_FS_16);`
— диапазон -16.. 16 g
- `setFullScaleGyroRange(MPU6050_GYRO_FS_250);`
— диапазон -250.. 250 град/с
- `setFullScaleGyroRange(MPU6050_GYRO_FS_500);`
— диапазон -500.. 500 град/с
- `setFullScaleGyroRange(MPU6050_GYRO_FS_1000);`
— диапазон -1000.. 1000 град/с
- `setFullScaleGyroRange(MPU6050_GYRO_FS_2000);`
— диапазон -2000.. 2000 град/с

```
void setup() {
  Wire.begin();
  mpu.initialize();
  mpu.setFullScaleAccelRange(MPU6050_ACCEL_FS_4);
  mpu.setFullScaleGyroRange(MPU6050_GYRO_FS_1000);
}
```

Калибровка

Пример из библиотеки

```
// положи датчик горизонтально, надписями вверх
// ДАЖЕ НЕ ДЫШИ НА НЕГО
// отправь любой символ в сериал, чтобы начать калибровку
// жди результатов

// ----- НАСТРОЙКИ -----
const int buffersize = 70;      // количество итераций калибровки
const int acel_deadzone = 10;   // точность калибровки акселерометра (по умолчанию 8)
const int gyro_deadzone = 6;    // точность калибровки гироскопа (по умолчанию 2)
// ----- НАСТРОЙКИ -----

#include "I2Cdev.h"
#include "MPU6050.h"
#include "Wire.h"
MPU6050 mpu(0x68);
int16_t ax, ay, az, gx, gy, gz;
int mean_ax, mean_ay, mean_az, mean_gx, mean_gy, mean_gz, state = 0;
int ax_offset, ay_offset, az_offset, gx_offset, gy_offset, gz_offset;

//////////////////////      SETUP      //////////////////////////////////////
```

```

void setup() {
  Wire.begin();
  Serial.begin(9600);
  mpu.initialize();
  // ждём очистки сериал
  while (Serial.available() && Serial.read()); // чистим
  while (!Serial.available()) {
    Serial.println(F("Send any character to start sketch.\n"));
    delay(1500);
  }
  while (Serial.available() && Serial.read()); // чистим ещё на всякий
  Serial.println("\nMPU6050 Calibration Sketch");
  delay(2000);
  Serial.println("\nYour MPU6050 should be placed in horizontal position, with
package letters facing up");
  delay(3000);

  // проверка соединения
  Serial.println(mpu.testConnection() ? "MPU6050 connection successful" : "MPU6050
connection failed");
  delay(1000);

  // сбросить оффсеты
  mpu.setXAccelOffset(0);
  mpu.setYAccelOffset(0);
  mpu.setZAccelOffset(0);
  mpu.setXGyroOffset(0);
  mpu.setYGyroOffset(0);
  mpu.setZGyroOffset(0);
}

//////////////////////////////////// LOOP //////////////////////////////////////
void loop() {
  if (state == 0) {
    Serial.println("\nReading sensors for first time...");
    meansensors();
    state++;
    delay(1000);
  }
  if (state == 1) {
    Serial.println("\nCalculating offsets...");
    calibration();
    state++;
    delay(1000);
  }
  if (state == 2) {
    meansensors();
    Serial.println("\nFINISHED!");
    Serial.print("\nSensor readings with offsets:\t");
    Serial.print(mean_ax);
    Serial.print("\t");
    Serial.print(mean_ay);
    Serial.print("\t");
  }
}

```



```

Serial.print(mean_az);
Serial.print("\t");
Serial.print(mean_gx);
Serial.print("\t");
Serial.print(mean_gy);
Serial.print("\t");
Serial.println(mean_gz);
Serial.print("Your offsets:\t");
Serial.print(ax_offset);
Serial.print(", ");
Serial.print(ay_offset);
Serial.print(", ");
Serial.print(az_offset);
Serial.print(", ");
Serial.print(gx_offset);
Serial.print(", ");
Serial.print(gy_offset);
Serial.print(", ");
Serial.println(gz_offset);
Serial.println("\nData is printed as: accelX accelY accelZ giroX giroY giroZ");
Serial.println("Check that your sensor readings are close to 0 0 16384 0 0
0");
Serial.println("If calibration was succesful write down your offsets so you
can set them in your projects using something similar to
mpu.setXAccelOffset(youroffset)");
while (1);
}
}
////////////////////////////////////// FUNCTIONS
//////////////////////////////////////
void meansensors() {
    long i = 0, buff_ax = 0, buff_ay = 0, buff_az = 0, buff_gx = 0, buff_gy = 0,
buff_gz = 0;
    while (i < (buffersize + 101)) { // read raw accel/gyro measurements from device
        mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
        if (i > 100 && i <= (buffersize + 100)) { //First 100 measures are discarded
            buff_ax = buff_ax + ax;
            buff_ay = buff_ay + ay;
            buff_az = buff_az + az;
            buff_gx = buff_gx + gx;
            buff_gy = buff_gy + gy;
            buff_gz = buff_gz + gz;
        }
        if (i == (buffersize + 100)) {
            mean_ax = buff_ax / buffersize;
            mean_ay = buff_ay / buffersize;
            mean_az = buff_az / buffersize;
            mean_gx = buff_gx / buffersize;
            mean_gy = buff_gy / buffersize;
            mean_gz = buff_gz / buffersize;
        }
        i++;
        delay(2);
    }
}

```

```

    }
}

void calibration() {
    ax_offset = -mean_ax / 8;
    ay_offset = -mean_ay / 8;
    az_offset = (16384 - mean_az) / 8;
    gx_offset = -mean_gx / 4;
    gy_offset = -mean_gy / 4;
    gz_offset = -mean_gz / 4;

    while (1) {
        int ready = 0;
        mpu.setXAccelOffset(ax_offset);
        mpu.setYAccelOffset(ay_offset);
        mpu.setZAccelOffset(az_offset);
        mpu.setXGyroOffset(gx_offset);
        mpu.setYGyroOffset(gy_offset);
        mpu.setZGyroOffset(gz_offset);
        meansensors();
        Serial.println("...");

        if (abs(mean_ax) <= accel_deadzone) ready++;
        else ax_offset = ax_offset - mean_ax / accel_deadzone;
        if (abs(mean_ay) <= accel_deadzone) ready++;
        else ay_offset = ay_offset - mean_ay / accel_deadzone;
        if (abs(16384 - mean_az) <= accel_deadzone) ready++;
        else az_offset = az_offset + (16384 - mean_az) / accel_deadzone;
        if (abs(mean_gx) <= gyro_deadzone) ready++;
        else gx_offset = gx_offset - mean_gx / (gyro_deadzone + 1);
        if (abs(mean_gy) <= gyro_deadzone) ready++;
        else gy_offset = gy_offset - mean_gy / (gyro_deadzone + 1);
        if (abs(mean_gz) <= gyro_deadzone) ready++;
        else gz_offset = gz_offset - mean_gz / (gyro_deadzone + 1);
        if (ready == 6) break;
    }
}

```

Встроенные функции

```

mpu.CalibrateAccel(6);
mpu.CalibrateGyro(6);

```

Получение углов

Асс + тригонометрия

```

#include "MPU6050.h"
MPU6050 mpu;

```

```

void setup() {
    Wire.begin();
    Serial.begin(9600);
    mpu.initialize();    // запускаем датчик
}

void loop() {
    int16_t ax = mpu.getAccelerationX(); // ускорение по оси X
    // стандартный диапазон: +-2g
    ax = constrain(ax, -16384, 16384);    // ограничиваем +-1g
    float angle = ax / 16384.0;           // переводим в +-1.0

    // и в угол через арксинус
    if (angle < 0) angle = 90 - degrees(acos(angle));
    else angle = degrees(acos(-angle)) - 90;

    Serial.println(angle);
    delay(5);
}

```

Также акселерометр шумит и показания плавают, не говоря о минимальном разрешении $\pm 2g$ и резком возникновении ошибки при ускорении модуля вдоль измеряемой оси, то есть угол покоящегося модуля он покажет, но стоит начать его двигать – угол изменится. Для точного определения угла использую показания ускорения в связке с угловой скоростью.

Для получения углов из показаний акселерометра и гироскопа есть три основных способа:

- Фильтрация при помощи фильтра Калмана.
- Фильтрация при помощи комплементарного фильтра.
- Определение углов при помощи встроенного в датчик DMP (Digital Motion Processor).

Из всех трёх самый хороший результат даёт встроенный процессор, на его фоне Калман и комплементарный можно даже не рассматривать, они практически не работают.

****Калман + комплементарный****

```

```cpp
#define COMPL_K 0.07
#include "Wire.h"
#include "Kalman.h"
Kalman kalmanX;
Kalman kalmanY;
uint8_t IMUAddress = 0x68;
/* IMU Data */
int16_t accX;
int16_t accY;
int16_t accZ;
int16_t tempRaw;
int16_t gyroX;
int16_t gyroY;
int16_t gyroZ;
float accXangle; // Angle calculate using the accelerometer
float accYangle;

```

```

float temp;
float gyroXangle = 180; // Angle calculate using the gyro
float gyroYangle = 180;
float compAngleX = 180; // Calculate the angle using a Kalman filter
float compAngleY = 180;
float kalAngleX; // Calculate the angle using a Kalman filter
float kalAngleY;
uint32_t timer;
void setup() {
 Serial.begin(9600);
 Wire.begin();
 Wire.beginTransmission(IMUAddress);
 Wire.write(0x6B); // PWR_MGMT_1 register
 Wire.write(0); // set to zero (wakes up the MPU-6050)
 Wire.endTransmission(true);
 kalmanX.setAngle(180); // Set starting angle
 kalmanY.setAngle(180);
 timer = micros();
}
void loop() {
 //uint32_t looptime = micros();
 getValues();
 calculateAngles();
 //Serial.println(micros() - looptime);
 Serial.print(kalAngleX); Serial.print(" ");
 Serial.print(kalAngleY); Serial.println();
 /*
 Serial.print(compAngleX); Serial.print(" ");
 Serial.print(compAngleY); Serial.println();
 */
 delay(20); // The accelerometer's maximum samples rate is 1kHz
}
void getValues() {
 /* Update all the values */
 Wire.beginTransmission(IMUAddress);
 Wire.write(0x3B); // starting with register 0x3B (ACCEL_XOUT_H)
 Wire.endTransmission(false);
 Wire.requestFrom(IMUAddress, 14, true); // request a total of 14 registers
 accX = Wire.read() << 8 | Wire.read(); // 0x3B (ACCEL_XOUT_H) & 0x3C
 (ACCEL_XOUT_L)
 accY = Wire.read() << 8 | Wire.read(); // 0x3D (ACCEL_YOUT_H) & 0x3E
 (ACCEL_YOUT_L)
 accZ = Wire.read() << 8 | Wire.read(); // 0x3F (ACCEL_ZOUT_H) & 0x40
 (ACCEL_ZOUT_L)
 tempRaw = Wire.read() << 8 | Wire.read(); // 0x41 (TEMP_OUT_H) & 0x42
 (TEMP_OUT_L)
 gyroX = Wire.read() << 8 | Wire.read(); // 0x43 (GYRO_XOUT_H) & 0x44
 (GYRO_XOUT_L)
 gyroY = Wire.read() << 8 | Wire.read(); // 0x45 (GYRO_YOUT_H) & 0x46
 (GYRO_YOUT_L)
 gyroZ = Wire.read() << 8 | Wire.read(); // 0x47 (GYRO_ZOUT_H) & 0x48
 (GYRO_ZOUT_L)
 //temp = ((float)tempRaw + 12412.0) / 340.0;

```

```

}
void calculateAngles() {
 /* Calculate the angles based on the different sensors and algorithm */
 accYangle = (atan2(accX, accZ) + PI) * RAD_TO_DEG;
 accXangle = (atan2(accY, accZ) + PI) * RAD_TO_DEG;
 float gyroXrate = (float)gyroX / 131.0;
 float gyroYrate = -((float)gyroY / 131.0);
 // Calculate gyro angle without any filter
 gyroXangle += gyroXrate * ((float)(micros() - timer) / 1000000);
 gyroYangle += gyroYrate * ((float)(micros() - timer) / 1000000);
 /*
 // Calculate gyro angle using the unbiased rate
 gyroXangle += kalmanX.getRate() * ((float)(micros() - timer) / 1000000);
 gyroYangle += kalmanY.getRate() * ((float)(micros() - timer) / 1000000);
 */
 /*
 // Calculate the angle using a Complimentary filter
 compAngleX = ((float)(1 - COMPL_K) * (compAngleX + (gyroXrate * (float)
(micros() - timer) / 1000000))) + (COMPL_K * accXangle);
 compAngleY = ((float)(1 - COMPL_K) * (compAngleY + (gyroYrate * (float)
(micros() - timer) / 1000000))) + (COMPL_K * accYangle);
 */

 // Calculate the angle using a Kalman filter
 kalAngleX = kalmanX.getAngle(accXangle, gyroXrate, (float)(micros() - timer) /
1000000);
 kalAngleY = kalmanY.getAngle(accYangle, gyroYrate, (float)(micros() - timer) /
1000000);
 timer = micros();
}

```

## Применение DMP

Пример получения углов при помощи DMP и встроенной библиотеки *MPU6050\_6Axis\_MotionApps20.h*. Единственное, скетч занимает прилично памяти (11кб Flash и 500б SRAM), так как фактически содержит прошивку для DMP и загружает её в модуль при каждом старте программы. С модуля можно подключить пин INT на один из пинов прерывания МК, например на D2 в случае с Arduino Nano (номер прерывания 0). DMP будет дёргать прерывание, сообщая о готовности, чтобы мы не тратили лишнее время на его постоянный опрос. Но опрашивать то можно и по таймеру (второй пример).

### Получение углов по прерыванию

```

#include "I2Cdev.h"
#include "MPU6050_6Axis_MotionApps20.h"
MPU6050 mpu;

volatile bool mpuFlag = false; // флаг прерывания готовности
uint8_t fifoBuffer[45]; // буфер

void setup() {

```

```

Serial.begin(115200);
Wire.begin();
//Wire.setClock(1000000UL); // разгоняем шину на максимум

// инициализация DMP и прерывания
mpu.initialize();
mpu.dmpInitialize();
mpu.setDMPEnabled(true);
attachInterrupt(0, dmpReady, RISING);
}

// прерывание готовности. Поднимаем флаг
void dmpReady() {
 mpuFlag = true;
}

void loop() {
 // по флагу прерывания и готовности DMP
 if (mpuFlag && mpu.dmpGetCurrentFIFOPacket(fifoBuffer)) {
 // переменные для расчёта (ypr можно вынести в глобал)
 Quaternion q;
 VectorFloat gravity;
 float ypr[3];

 // расчёты
 mpu.dmpGetQuaternion(&q, fifoBuffer);
 mpu.dmpGetGravity(&gravity, &q);
 mpu.dmpGetYawPitchRoll(ypr, &q, &gravity);
 mpuFlag = false;

 // выводим результат в радианах (-3.14, 3.14)
 Serial.print(ypr[0]); // вокруг оси Z
 Serial.print(',');
 Serial.print(ypr[1]); // вокруг оси Y
 Serial.print(',');
 Serial.print(ypr[2]); // вокруг оси X
 Serial.println();
 // для градусов можно использовать degrees()
 }
}

```

## Использование таймера

```

#include "I2Cdev.h"
#include "MPU6050_6Axis_MotionApps20.h"
MPU6050 mpu;

uint8_t fifoBuffer[45]; // буфер

void setup() {
 Serial.begin(115200);

```

```

Wire.begin();
//Wire.setClock(1000000UL); // разгоняем шину на максимум

// инициализация DMP
mpu.initialize();
mpu.dmpInitialize();
mpu.setDMPEnabled(true);
}

void loop() {
 static uint32_t tmr;
 if (millis() - tmr >= 11) { // таймер на 11 мс (на всякий случай)
 if (mpu.dmpGetCurrentFIFOPacket(fifoBuffer)) {
 // переменные для расчёта (ypr можно вынести в глобал)
 Quaternion q;
 VectorFloat gravity;
 float ypr[3];

 // расчёты
 mpu.dmpGetQuaternion(&q, fifoBuffer);
 mpu.dmpGetGravity(&gravity, &q);
 mpu.dmpGetYawPitchRoll(ypr, &q, &gravity);

 // выводим результат в радианах (-3.14, 3.14)
 Serial.print(ypr[0]); // вокруг оси Z
 Serial.print(',');
 Serial.print(ypr[1]); // вокруг оси Y
 Serial.print(',');
 Serial.print(ypr[2]); // вокруг оси X
 Serial.println();
 // для градусов можно использовать degrees()

 tmr = millis(); // сброс таймера
 }
 }
}

```